

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5

7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I V. Sujithra (192521445) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Apply Relational Algebra Operations on Employee and Department Relations

Consider the following relations:

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Arun	D01	60000
102	Ravi	D02	45000
103	Priya	D01	70000
104	Kumar	D03	50000

DEPARTMENT

DeptID	DeptName
D01	IT
D02	HR
D03	Finance

Relational algebra is a **procedural query language** used to retrieve data from relations.

1. Selection (σ)

Selection is used to select rows satisfying a condition.

Find employees belonging to D01:

Result:

EmpID	EmpName	DeptID	Salary
101	Arun	D01	60000
103	Priya	D01	70000

2. Projection (π)

Projection is used to select specific columns.

Display employee names and salaries:

3. Union (U)

Union combines tuples from two **union-compatible relations**.

It returns all records from both relations without duplicate tuples.

4. Set Difference (-)

It returns tuples present in the first relation but not in the second.

5. Cartesian Product (×)

It combines every tuple of one relation with every tuple of another relation.

6. Join (⋈)

To combine employee details with department details:

Result contains:

EmpName	DeptName	Salary
Arun	IT	60000
Ravi	HR	45000
Priya	IT	70000
Kumar	Finance	50000

Conclusion

Thus, σ , π , U , $-$, $\times$ and $\Join$ operations can be used to retrieve and combine required information from Employee and Department relations.

Q2. Write SQL DDL Statements to Create STUDENT and DEPARTMENT Tables

DDL stands for **Data Definition Language**. It is used to create and modify database structures.

The important DDL commands are CREATE, ALTER, DROP and TRUNCATE.

Consider two tables:

- STUDENT
- DEPARTMENT

Step 1: Create DEPARTMENT table

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);
```

Step 2: Create STUDENT table

```
CREATE TABLE STUDENT (  
    SID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL,  
    DeptID INT,  
    FOREIGN KEY (DeptID)  
        REFERENCES DEPARTMENT(DeptID)  
);
```

Constraints Used

1. PRIMARY KEY

SID INT PRIMARY KEY

It uniquely identifies each student.

2. NOT NULL

Name VARCHAR(50) NOT NULL

It prevents empty values.

3. UNIQUE

DeptName VARCHAR(50) UNIQUE

It does not allow duplicate department names.

4. FOREIGN KEY

FOREIGN KEY (DeptID)

REFERENCES DEPARTMENT(DeptID)

It establishes a relationship between STUDENT and DEPARTMENT.

ALTER Example

To add a phone number:

```
ALTER TABLE STUDENT
```

```
ADD Phone VARCHAR(15);
```

Conclusion

Thus, DDL statements can be used to create STUDENT and DEPARTMENT tables with appropriate **primary key, foreign key, NOT NULL and UNIQUE constraints**.

Q3. SQL Queries Using GROUP BY, HAVING and Aggregate Functions

Aggregate functions perform calculations on a group of rows.

Common aggregate functions are:

- COUNT() – counts records
- SUM() – calculates total
- AVG() – calculates average
- MAX() – finds maximum
- MIN() – finds minimum

Consider:

SALES

SaleID	Product	Category	Amount
1	Laptop	Electronics	50000
2	Phone	Electronics	30000
3	Chair	Furniture	10000
4	Table	Furniture	20000
5	TV	Electronics	40000

1. COUNT()

Find the number of sales in each category.

```
SELECT Category, COUNT(*) AS TotalSales
```

```
FROM SALES
```

```
GROUP BY Category;
```

2. SUM()

Find the total sales amount for each category.

```
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category;
```

3. AVG()

Find the average sales amount.

```
SELECT Category, AVG(Amount) AS AverageAmount
FROM SALES
GROUP BY Category;
```

4. MAX() and MIN()

```
SELECT Category,
    MAX(Amount) AS MaximumAmount,
    MIN(Amount) AS MinimumAmount
FROM SALES
GROUP BY Category;
```

5. HAVING

HAVING is used to filter groups after GROUP BY.

Find categories whose total sales are greater than 50,000:

```
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 50000;
```

Difference between WHERE and HAVING

- WHERE filters individual rows.

- HAVING filters groups.

Conclusion

Thus, GROUP BY, HAVING and aggregate functions are useful for performing **summary and statistical operations** on a dataset.

Q4. Correlated Subquery to Find Employees Earning More Than Department Average

A **correlated subquery** is a subquery that depends on the outer query. The inner query is executed for each row of the outer query.

Consider:

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Arun	10	60000
102	Ravi	10	45000
103	Priya	20	70000
104	Kumar	20	50000
105	Anu	10	55000

Query

```
SELECT E.EmpID,
       E.EmpName,
       E.DeptID,
       E.Salary
FROM EMPLOYEE E
WHERE E.Salary >
(
  SELECT AVG(E2.Salary)
  FROM EMPLOYEE E2
  WHERE E2.DeptID = E.DeptID
);
```

Explanation

1. The outer query selects employees from the EMPLOYEE table.
2. The inner query calculates the average salary of the employee's department.
3. E2.DeptID = E.DeptID makes the subquery correlated with the outer query.
4. The employee salary is compared with the department average.
5. Employees whose salary is greater than the department average are displayed.

Example Calculation

For Department 10:

Therefore, Arun and Anu earn more than the department average.

For Department 20:

Therefore, Priya earns more than the department average.

Expected Result

EmpID	EmpName	DeptID	Salary
101	Arun	10	60000
103	Priya	20	70000
105	Anu	10	55000

Conclusion

Thus, a correlated subquery is used to compare each employee's salary with the **average salary of their own department**.

Q5. Apply All Types of JOIN Operations

Consider:

EMPLOYEE

EmpID	EmpName	DeptID
101	Arun	D01
102	Ravi	D02
103	Priya	D03
104	Kumar	NULL

PROJECT

ProjectID	ProjectName	EmpID
P01	Banking App	101
P02	Web Portal	102
P03	AI System	105

1. INNER JOIN

Returns only matching records from both tables.

```
SELECT E.EmpName, P.ProjectName
```

```
FROM EMPLOYEE E
```

```
INNER JOIN PROJECT P
```

```
ON E.EmpID = P.EmpID;
```

It displays employees who have matching projects.

2. LEFT OUTER JOIN

Returns **all employees** and matching project details.

```
SELECT E.EmpName, P.ProjectName
```

```
FROM EMPLOYEE E
```

```
LEFT OUTER JOIN PROJECT P
```

```
ON E.EmpID = P.EmpID;
```

Even employees without projects are displayed.

3. RIGHT OUTER JOIN

Returns **all projects** and matching employee details.

```
SELECT E.EmpName, P.ProjectName
```

```
FROM EMPLOYEE E
```

```
RIGHT OUTER JOIN PROJECT P
```

```
ON E.EmpID = P.EmpID;
```

Even projects without matching employees are displayed.

4. FULL OUTER JOIN

Returns all matching and non-matching records from both tables.

```
SELECT E.EmpName, P.ProjectName  
FROM EMPLOYEE E  
FULL OUTER JOIN PROJECT P  
ON E.EmpID = P.EmpID;
```

5. CROSS JOIN

It produces the Cartesian product.

```
SELECT E.EmpName, P.ProjectName  
FROM EMPLOYEE E  
CROSS JOIN PROJECT P;
```

If there are 4 employees and 3 projects:
rows will be generated.

Conclusion

JOIN operations are used to retrieve related information from multiple tables. The major joins are **INNER**, **LEFT OUTER**, **RIGHT OUTER**, **FULL OUTER** and **CROSS JOIN**.

Q6. Create a View DeptSalaryView

A **view** is a virtual table created from one or more base tables. It does not normally store the actual data separately.

Consider:

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

DEPARTMENT(DeptID, DeptName)

The requirement is to display:

- Department name
- Total number of employees
- Average salary

Create View

```
CREATE VIEW DeptSalaryView AS  
  
SELECT D.DeptName,  
       COUNT(E.EmpID) AS TotalEmployees,  
       AVG(E.Salary) AS AverageSalary  
  
FROM DEPARTMENT D  
  
LEFT JOIN EMPLOYEE E  
  
ON D.DeptID = E.DeptID  
  
GROUP BY D.DeptID, D.DeptName;
```

Query the View

```
SELECT *  
  
FROM DeptSalaryView;
```

Example Output

DeptName	TotalEmployees	AverageSalary
IT	5	58000
HR	3	45000
Finance	4	52000

Query a Particular Department

```
SELECT *  
  
FROM DeptSalaryView  
  
WHERE DeptName = 'IT';
```

Advantages of View

1. Provides simplified access to complex queries.
2. Improves data security.
3. Hides unnecessary columns.
4. Can be reused like a table.
5. Makes reporting easier.

Conclusion

Thus, DeptSalaryView provides department-wise **employee count and average salary** in a simple virtual table.

Q7. Relational Algebra – Students Enrolled in All Courses

Consider the following relations:

STUDENT

SID	SName
101	Arun
102	Ravi
103	Priya

COURSE

CID	CourseName
C01	DBMS
C02	OS
C03	CN

ENROLL

SID	CID
101	C01
101	C02
101	C03
102	C01
102	C02
103	C01

The question asks to find students who are enrolled in **all courses offered**.

For this type of question, **Relational Division ($\div$)** is used.

Step 1: Perform Division

This returns the SID values of students who are enrolled in every course.

Result

SID

101

Student 101 has enrolled in C01, C02 and C03.

Step 2: Join with STUDENT

Final Projection

Final Result

SName

Arun

Explanation

Relational division is used when the requirement contains words such as:

- **all**
- **every**
- **each**

Here, the requirement is "students enrolled in **all courses**", so relational division is appropriate.

Conclusion

Thus,

retrieves the names of students enrolled in all the courses offered.

Q8. SQL DML Statements – INSERT, UPDATE and DELETE

DML stands for **Data Manipulation Language**. It is used to insert, modify and delete data from tables.

The major DML commands are:

- INSERT
- UPDATE
- DELETE

Consider:

STUDENT(SID, Name, DOB, DeptID)

1. INSERT

The INSERT command is used to add new records.

INSERT INTO STUDENT

(SID, Name, DOB, DeptID)

VALUES

(101, 'Arun', '2005-05-10', 10);

Multiple Records

INSERT INTO STUDENT

(SID, Name, DOB, DeptID)

VALUES

(102, 'Ravi', '2005-06-15', 20),

(103, 'Priya', '2005-08-20', 10);

2. UPDATE

The UPDATE command modifies existing records.

Change student 101's department:

UPDATE STUDENT

SET DeptID = 20

WHERE SID = 101;

The WHERE condition is important because it identifies which record must be updated.

3. DELETE

The DELETE command removes records.

DELETE FROM STUDENT

WHERE SID = 103;

Only student 103 is deleted.

Integrity Constraint Enforcement

Suppose SID is a primary key.

INSERT INTO STUDENT

VALUES (101, 'Kumar', '2005-01-01', 10);

If SID 101 already exists, the DBMS rejects the operation because a **primary key cannot contain duplicate values**.

Similarly, if DeptID is a foreign key and we insert a department ID that does not exist in DEPARTMENT, the DBMS rejects the record.

Conclusion

Thus, DML commands are used to **insert, update and delete records**, while integrity constraints ensure that invalid data is not stored.

Q9. Tuple Relational Calculus

Question:

Find all employees who work in the IT department and earn more than ₹50,000.

Consider:

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

Tuple Relational Calculus (TRC) is a **non-procedural query language**. It describes **what data is required**, rather than how to retrieve it.

TRC Expression

Explanation

Here:

- e represents an employee tuple.
- d represents a department tuple.
- EMPLOYEE(e) ensures that e is an employee.
- DEPARTMENT(d) ensures that d is a department.
- e.DeptID = d.DeptID connects employee and department.

- d.DeptName = 'IT' selects the IT department.
- e.Salary > 50000 selects employees earning more than ₹50,000.

Example

Suppose:

EmpID	EmpName	DeptID	Salary
101	Arun	D01	60000
102	Ravi	D02	70000
103	Priya	D01	45000
104	Kumar	D01	80000

And D01 = IT.

The result is:

EmpID	EmpName	Salary
101	Arun	60000
104	Kumar	80000

Conclusion

Thus, TRC can express the required query by specifying the **conditions that the required employee tuples must satisfy**.

Q10. Airline Reservation System – Relational Schema and SQL Queries

An Airline Reservation System manages passengers, flights, bookings, aircraft and payments.

Relational Schema

1. PASSENGER

PASSENGER(

 PassengerID,

 Name,

 Phone,

 Email

)

PassengerID is the Primary Key.

2. AIRCRAFT

AIRCRAFT(

AircraftID,

Model,

Capacity

)

AircraftID is the Primary Key.

3. FLIGHT

FLIGHT(

FlightID,

Source,

Destination,

DepartureTime,

ArrivalTime,

AircraftID

)

FlightID is the Primary Key.

AircraftID is a Foreign Key referencing AIRCRAFT.

4. BOOKING

BOOKING(

BookingID,

PassengerID,

FlightID,

BookingDate,

SeatNo

)

BookingID is the Primary Key.

PassengerID and FlightID are Foreign Keys.

5. PAYMENT

PAYMENT(

PaymentID,

BookingID,

Amount,

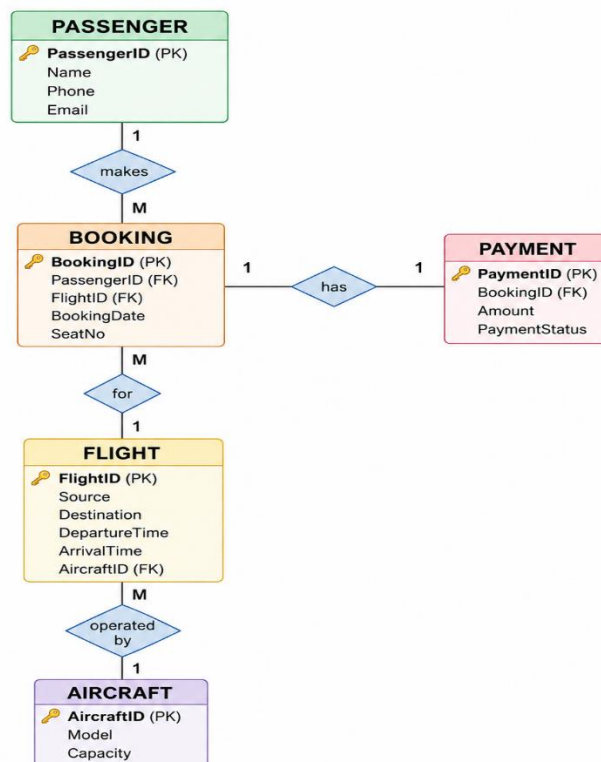
PaymentStatus

)

PaymentID is the Primary Key.

BookingID is a Foreign Key.

Relationships



Query 1 – JOIN

Display passenger names and their flight details.

```
SELECT P.Name,  
       F.Source,  
       F.Destination  
FROM PASSENGER P  
JOIN BOOKING B  
ON P.PassengerID = B.PassengerID  
JOIN FLIGHT F  
ON B.FlightID = F.FlightID;
```

This demonstrates **multiple table joins**.

Query 2 – JOIN with Aircraft

Display flight details along with aircraft model.

```
SELECT F.FlightID,  
       F.Source,  
       F.Destination,  
       A.Model  
FROM FLIGHT F  
JOIN AIRCRAFT A  
ON F.AircraftID = A.AircraftID;
```

Query 3 – Subquery

Find passengers who booked flights going to Chennai.

```
SELECT Name  
FROM PASSENGER
```

```
WHERE PassengerID IN
(
    SELECT PassengerID
    FROM BOOKING
    WHERE FlightID IN
    (
        SELECT FlightID
        FROM FLIGHT
        WHERE Destination = 'Chennai'
    )
);
```

This demonstrates a **nested subquery**.

Query 4 – View

Create a view that displays passenger booking information.

```
CREATE VIEW BookingDetails AS
```

```
SELECT P.Name,
```

```
       F.Source,
```

```
       F.Destination,
```

```
       B.SeatNo,
```

```
       B.BookingDate
```

```
FROM PASSENGER P
```

```
JOIN BOOKING B
```

```
ON P.PassengerID = B.PassengerID
```

```
JOIN FLIGHT F
```

```
ON B.FlightID = F.FlightID;
```

To display the view:

```
SELECT *
```

```
FROM BookingDetails;
```

Query 5 – GROUP BY

Find the total number of bookings for each flight.

```
SELECT FlightID,  
       COUNT(*) AS TotalBookings  
FROM BOOKING  
GROUP BY FlightID;
```

This demonstrates the use of **GROUP BY** and **COUNT()**.

Conclusion

The Airline Reservation System uses multiple related tables such as **PASSENGER**, **FLIGHT**, **AIRCRAFT**, **BOOKING** and **PAYMENT**. SQL joins, subqueries, views and aggregate functions can be used to retrieve useful reservation information efficiently.